Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий

Учебная дисциплина «Метрология, стандартизация и сертификация (в информационных технологиях)»

ОТЧЁТ
по лабораторной работе №3
«Метрики сложности потока управления программ»

Подготовил: Зданович П. А.
ст. гр. 410901

Проверил: Болтак С. В.

Минск 2026

1. Код анализируемой программы

Язык программирования — Groovy. Программа обрабатывает ведомость оценок группы. Вводятся количество студентов n, количество предметов m и матрица оценок marks[n][m]. Программа считает сумму всех оценок total и количество задолженностей debts (оценок ниже 4). По средней оценке avg определяется уровень успеваемости группы, он выводится на экран вместе с total и debts.

Каждый цикл for на схеме изображён тремя блоками: инициализация переменной цикла, проверка условия и изменение переменной цикла. Номера блоков схемы указаны в комментариях к коду.

```
def sc = new Scanner(System.in)                                    // блок 2
int n = sc.nextInt()                                               // блок 3
int m = sc.nextInt()                                               // блок 4
int[][] marks = new int[n][m]                                      // блок 5
int total = 0                                                      // блок 6
for (int i = 0; i < n; i++) {                                      // блоки 7, 8, 14
    for (int j = 0; j < m; j++) {                                  // блоки 9, 10, 13
        marks[i][j] = sc.nextInt()                                 // блок 11
        total += marks[i][j]                                       // блок 12
    }
}
int debts = 0                                                      // блок 15
for (int i = 0; i < n; i++) {                                      // блоки 16, 17, 23
    for (int j = 0; j < m; j++) {                                  // блоки 18, 19, 22
        if (marks[i][j] < 4) {                                     // блок 20
            debts++                                                // блок 21
        }
    }
}
int avg = total.intdiv(n * m)                                      // блок 24
int level = avg.intdiv(4)                                          // блок 25
switch (level) {                                                   // блок 26
    case 0:
        println("Низкий уровень")                                  // блок 27
        break                                                      // блок 28
    case 1:                                                        // блок 29
        println("Средний уровень")                                 // блок 30
        break                                                      // блок 31
    default:
        println("Высокий уровень")                                 // блок 32
}
println(total)                                                     // блок 33
println(debts)                                                     // блок 34
                                                   // блок 35 (Конец)
```

2. Схема алгоритма

Схема алгоритма детализированная: каждому блоку соответствует один оператор программы. Ввод двумерного массива представлен двумя вложенными циклами, каждый цикл изображён с инициализацией и изменением переменной цикла. Оператор множественного выбора switch изображён в виде вложенных друг в друга операторов условия. Для переноса схемы на следующий лист использованы межстраничные соединители «А» и «Б». Номер блока указан над его левым верхним углом.

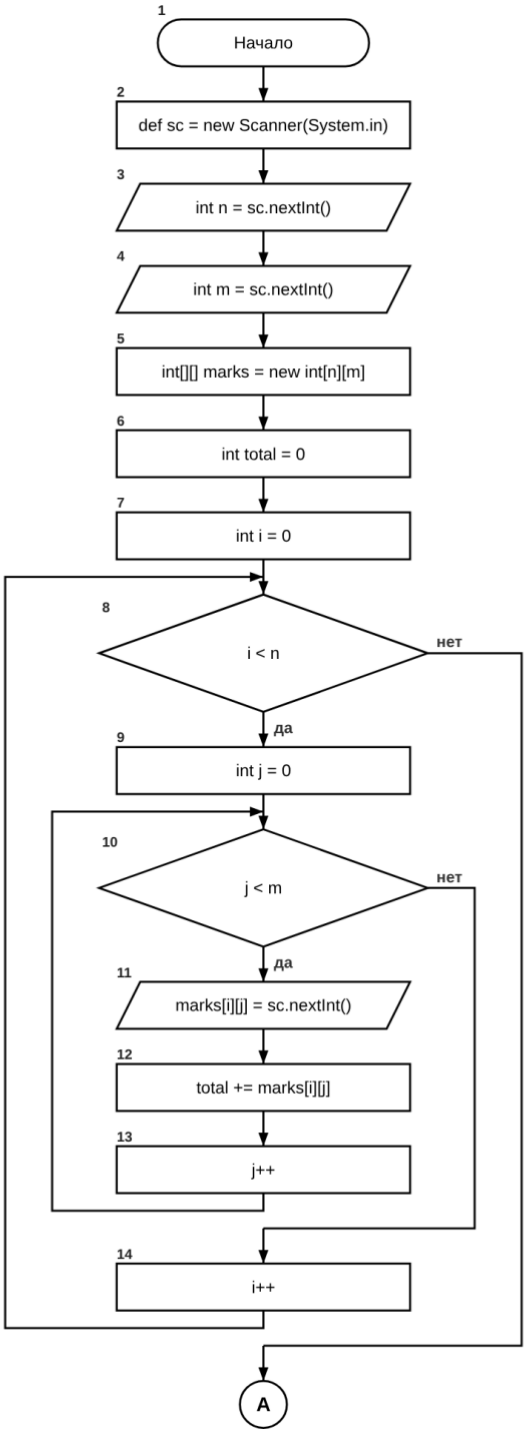


Рисунок 1 — Схема алгоритма программы (лист 1)

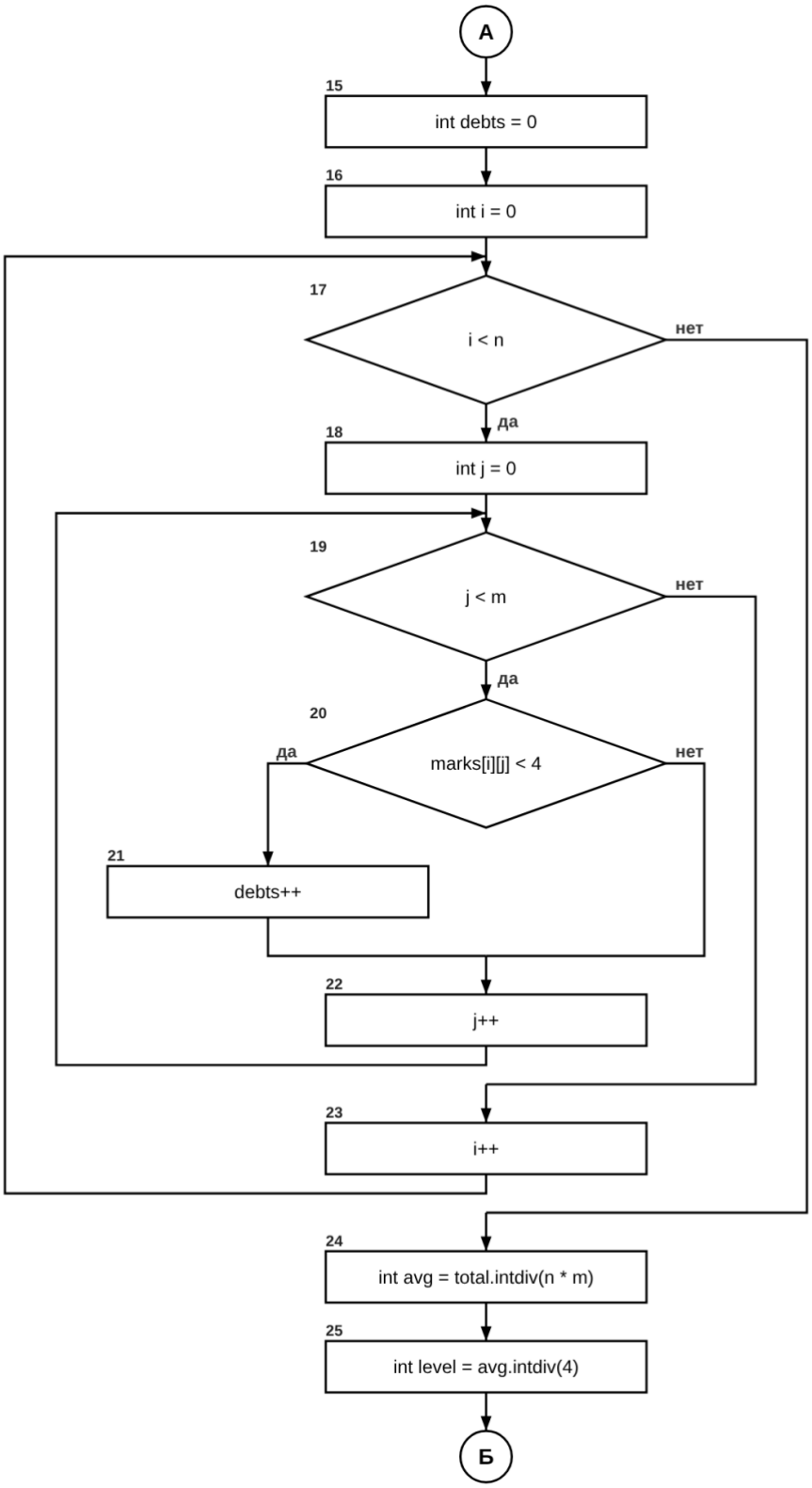


Рисунок 1 — Схема алгоритма программы (лист 2)

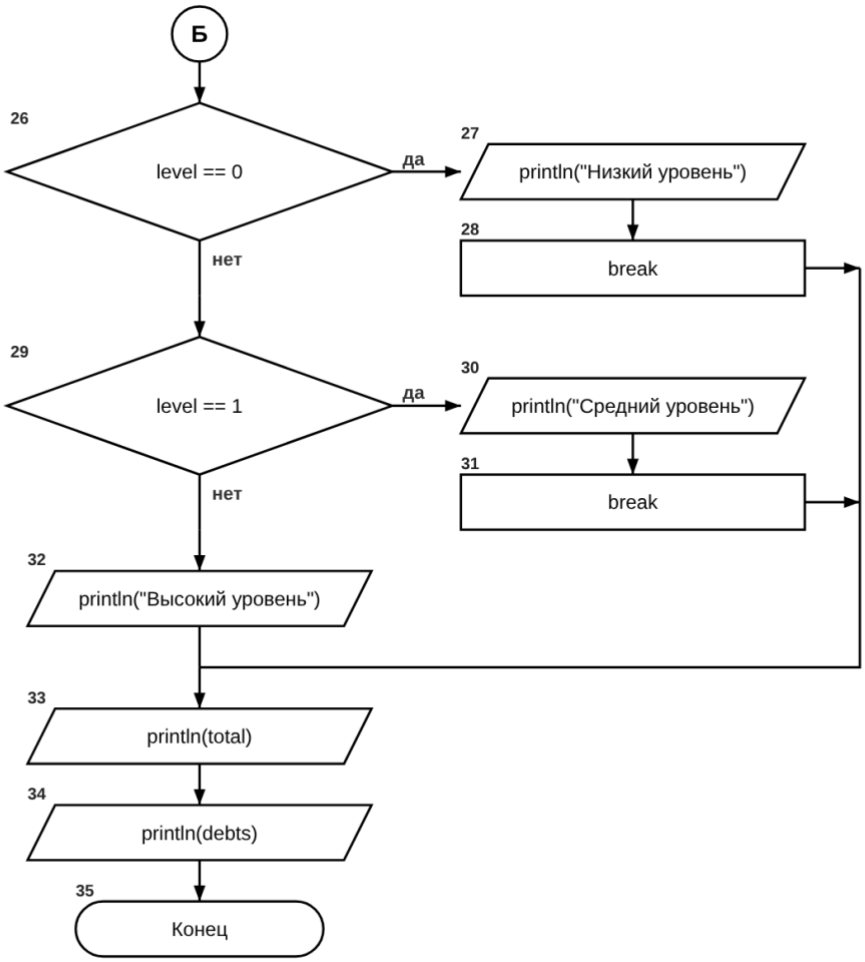


Рисунок 1 — Схема алгоритма программы (лист 3)

3. Результаты расчёта метрики граничных значений

Граф программы построен по схеме алгоритма (рис. 1): вершины графа соответствуют блокам схемы, номера вершин совпадают с номерами блоков. Соединители «А» и «Б» вершинами не являются: они только переносят линию на другой лист. Общее число вершин графа $\upsilon = 35$.

Вершины выбора (положительная степень ≥ 2): 8, 10, 17, 19, 20, 26, 29 — всего 7 вершин. Остальные вершины, кроме конечной, являются принимающими и имеют скорректированную сложность, равную 1. Конечная вершина 35 имеет скорректированную сложность 0.

Скорректированная сложность вершины выбора равна числу вершин её подграфа вместе с нижней границей подграфа. Для вершин выбора циклов (8, 10, 17, 19) в подграф входит и сама вершина выбора, так как тело цикла возвращает управление в неё. У вершины 20 (if без else) ветка «нет» сразу ведёт в вершину 22, поэтому вершина 22 является нижней границей подграфа. Ветви case оператора switch (вершины 26 и 29) заканчиваются оператором break и сходятся в вершине 33.

Таблица 1 — Свойства подграфов программы (рис. 1), вершины выбора 8–19

Свойства подграфов программы	**Номер вершины выбора**			
	8	**10**	**17**	**19**
Номера вершин перехода	9, 15	11, 14	18, 24	20, 23
Скорректированная сложность вершины выбора	8	5	8	5
Номера вершин подграфа	8–14	10–13	17–23	19–22
Номер нижней границы подграфа	15	14	24	23

Таблица 2 — Свойства подграфов программы (рис. 1), вершины выбора 20–29

Свойства подграфов программы	**Номер вершины выбора**		
	20	**26**	**29**
Номера вершин перехода	21, 22	27, 29	30, 32
Скорректированная сложность вершины выбора	2	7	4

Номера вершин подграфа	21	27–32	30–32
Номер нижней границы подграфа	22	33	33

Таблица 3 — Скорректированные сложности вершин графа программы (рис. 1)

Номер вершины графа программы	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Скорректированная сложность вершины графа	1	1	1	1	1	1	1	8	1	5	1	1

Номер вершины графа программы	**13**	**14**	**15**	**16**	**17**	**18**	**19**	**20**	**21**	**22**	**23**	**24**
Скорректированная сложность вершины графа	1	1	1	1	8	1	5	2	1	1	1	1

Номер вершины графа программы	**25**	**26**	**27**	**28**	**29**	**30**	**31**	**32**	**33**	**34**	**35**
Скорректированная сложность вершины графа	1	7	1	1	4	1	1	1	1	1	0

Абсолютная граничная сложность программы равна сумме скорректированных сложностей всех вершин графа:

$$Sa = 27 \cdot 1 + (8 + 5 + 8 + 5 + 2 + 7 + 4) + 0 = 27 + 39 = 66.$$

Здесь 27 — количество принимающих вершин (35 вершин − 7 вершин выбора − 1 конечная вершина).

Относительная граничная сложность программы:

$$So = 1 - (\upsilon - 1) / Sa = 1 - (35 - 1) / 66 = 0{,}485.$$

Таблица 4 — Метрика граничных значений (рис. 1)

Метрика	**Значение**
Число вершин графа υ	35
Число вершин выбора	7
Метрика граничных значений (абсолютная граничная сложность программы) Sa	66
Метрика граничных значений (относительная граничная сложность программы) So	0,485

Таким образом, абсолютная граничная сложность программы Sa = 66, относительная граничная сложность So = 0,485.